DATA SCIENCE MINOR PROJECT REPORT

(Project Semester August-December 2026)

TELCO CUSTOMER CHURN PREDICTION USING MACHINE LEARNING

Submitted by

Raj Anand

Registration No: 12400521

Programme and Section: B.Tech Computer Science and Engineering, K2P24QR

Course Code: INT234 (Data Science Minor Project)

Under the Guidance of

Dr. Mrinalini Rana

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that **Raj Anand** bearing Registration no. **12400521** has completed the **INT234** project titled **"Telco Customer Churn Prediction Using Machine Learning"** under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

Signature and Name of the Supervisor: Dr. Mrinalini Rana

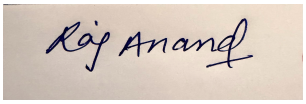
Designation of the Supervisor: Assistant Professor

School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date: 31/10/2026

DECLARATION

I, **Raj Anand**, student of **B.Tech Computer Science and Engineering** under the CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine. The dataset used was taken from a public source, which is named in Chapter 2, and all other sources I used are listed in the references.

Date: 31/10/2026 *Signature:* 

Registration No: 12400521 Name of the Student: Raj Anand

ACKNOWLEDGEMENT

I would like to thank my faculty coordinator and project supervisor, Dr. Mrinalini Rana, for the guidance and patience throughout this project. The questions raised during our discussions, especially about why I was removing certain columns and why one model was beating another, made me go back and check my own work more carefully than I otherwise would have.

I am also grateful to the Discipline of Computer Science and Engineering and to Lovely Professional University, Phagwara, for the lab facilities and the learning environment that made it possible to work on a project like this.

My thanks also go to IBM for publishing the Telco Customer Churn sample data, since the project would not have been possible without open data. Finally, I want to thank my friends and family for their support and for listening to me talk about confusion matrices for far longer than anyone should have to.

TABLE OF CONTENTS

Certificate ii
Declaration iii
Acknowledgement iv
List of Figures vi
List of Tables vii
1. Introduction 1
1.1 Background and Motivation 1
1.2 Problem Statement 1
1.3 Project Objectives 2
1.4 Tools and Environment 2
1.5 How This Report Is Organised 3
2. Source of Dataset 4
2.1 Where the Data Comes From 4
2.2 What Each Column Means 4
2.3 Class Balance and First Impressions 6
3. EDA Process 7
3.1 Exploring the Data 7
3.2 Which Customers Churn More 11
3.3 Tenure, Monthly Charge and Churn 15
3.4 Preparing Features for Machine Learning 20
4. Analysis on Dataset 25
4.1 Analysis 1: Logistic Regression 26
4.2 Analysis 2: K-Nearest Neighbors (KNN) 27
4.3 Analysis 3: Gaussian Naive Bayes 29
4.4 Analysis 4: Support Vector Machine (SVM) 30
4.5 Analysis 5: Benchmarking the Four Models 32
5. Conclusion 37
5.1 What the Project Found 37
5.2 What I Would Tell a Telecom Retention Team 38
5.3 Limitations 38
5.4 Project Links 38
6. Future Scope 40
7. References 41

LIST OF FIGURES

Figure 1: Churned versus stayed customers 8
Figure 2: Distribution of tenure in months 9
Figure 3: Distribution of monthly charge 10
Figure 4: Churn rate by contract type 12
Figure 5: Churn rate by internet type 13
Figure 6: Churn rate by Premium Tech Support 13
Figure 7: Churn rate by promotional offer 14
Figure 8: Churn rate by payment method 14
Figure 9: Tenure of churned and stayed customers 16
Figure 10: Monthly charge of churned and stayed customers 16
Figure 11: Monthly charge by contract type (stayed versus churned) 17
Figure 12: Churn rate (%) by tenure in months 18
Figure 13: Scatter plot of tenure versus monthly charge, coloured by churn 18
Figure 14: Pair plot of Age, Tenure, Monthly Charge and Total Charges, coloured by churn 19
Figure 15: Churn rate by satisfaction score (1.0 = 100%) 20
Figure 16: Correlation heat map of the main numeric columns 23
Figure 17: Confusion matrix of Logistic Regression on the test set 27
Figure 18: Confusion matrix of KNN (K = 5) on the test set 28
Figure 19: Confusion matrix of Gaussian Naive Bayes on the test set 30
Figure 20: Confusion matrix of the linear SVM on the test set 31
Figure 21: F1-score of the four models 34
Figure 22: Recall of the four models 34
Figure 23: AUC of the four models 34
Figure 24: ROC curves of the four models 35
Figure 25: GitHub post sharing the project 41
Figure 26: LinkedIn post sharing the project 42

LIST OF TABLES

Table 1: Feature dictionary of the Telco churn dataset (50 columns) 4
Table 2: Missing values per column and how I treated them 10
Table 3: Approximate churn rate by customer group (read from the bar charts) 15
Table 4: Approximate box-plot statistics (read from the plots) 16
Table 5: Churn rate by satisfaction score (read from the bar chart) 20
Table 6: Columns removed before modelling 21
Table 7: Encoding techniques used 22
Table 8: Final performance of all four models on the 1,409 test customers 33
Table 9: Confusion matrix counts for each model (test set, 1,409 customers) 33
Table 10: What each model would mean for a retention campaign (374 real leavers in the test set) 33

1. Introduction

1.1 Background and Motivation

A telecom company sells something that every customer can compare in a few minutes. Mobile, broadband and TV providers offer almost the same products, prices are posted online, and switching to another provider usually takes one phone call. In a market like this, the real battle is not only to win new customers but to keep the ones who are already there. Winning a new customer costs advertising, sales commission, installation visits and introductory discounts, while keeping an old customer mostly costs good service and a little attention at the right time.

When a customer stops using the services of a company, it is called churn. The difficulty is that churn is only visible after it has happened. By the time the cancellation request arrives, the customer has already decided, and the discounts and phone calls that might have saved the relationship come too late. If the company could know a few months earlier which customers are likely to leave, it could offer them a better plan, fix a service problem or simply call to ask whether everything is fine.

That is the idea behind predictive analytics, and it is the idea behind this project. A company already stores a lot of information about each customer: the services taken, the contract, the way of payment, the monthly bill and how many months the customer has stayed. I wanted to see how far ordinary machine learning could go in predicting churn from the kind of data a telecom company already keeps.

1.2 Problem Statement

The dataset I worked with describes 7,043 telecom customers through 50 columns covering personal details, services, contract and billing, charges, satisfaction and the final status, plus one target column called Churn Label that says whether the customer left. The problem is to turn this raw table into a trustworthy prediction of churn. The business question is simple: which telecom customers are likely to leave the company? In practice this meant answering five questions in order:

1. How many customers actually leave, and is the data balanced?
2. Which contract types, and which other customer groups, have the highest churn rate?
3. Are tenure and monthly charges related to churn?

4. Can machine learning models predict which customers will leave?
5. Which of the four models predicts churn most effectively?

The answers also have to be honest. Some columns in this dataset, such as Satisfaction Score and Churn Reason, are recorded after the customer has already decided to leave, and using them would make any model look perfect without being useful. Part of the problem was therefore to find and remove such columns before modelling.

1.3 Project Objectives

The work is organised around five objectives. I wrote them at the start of the project and they are also written as comments in my Python script, so every chapter of this report lines up with one of them. Whenever an objective is being addressed, it is repeated in a highlighted box, and the code that carries it out is placed right below.

> **Objective 1: Find the Percentage of Customers Who Leave**
> Find the percentage of customers who leave the company by counting how many customers churned and how many stayed.

> **Objective 2: Identify the Contract Types with the Highest Churn**
> Identify which contract types have the highest churn rate, and compare churn across internet type, premium tech support, offers and payment method.

> **Objective 3: Study Tenure and Monthly Charges**
> Investigate whether customer tenure and monthly charges are related to churn.

> **Objective 4: Predict Customer Churn**
> Train four classification models (Logistic Regression, KNN, Naive Bayes, SVM) to predict which customers are likely to leave.

> **Objective 5: Benchmark Model Performance**
> Compare Accuracy, Precision, Recall, F1-score and ROC-AUC of all four models to determine which one predicts churn most effectively.

Objectives 1 to 3 are covered in Chapter 3 (the EDA process, including cleaning and preparation). Objectives 4 and 5 are covered in Chapter 4, where each model is analysed separately and then all four are compared.

1.4 Tools and Environment

Everything in this project was written in Python and run in the Spyder IDE on a Mac. The libraries and tools I used are listed below.

- **Python 3** as the programming language for the whole project.
- **Spyder** as the development environment, because its plots pane and variable explorer made it easy to inspect every chart.
- **Pandas** for reading the CSV file, checking missing values, filling blanks, grouping, and one-hot encoding with get_dummies.
- **NumPy** for numerical operations and for the mean estimator used in the bar charts.
- **Matplotlib** for the pie chart, histograms, line chart and ROC curves.
- **Seaborn** for the bar plots, box plots, scatter plot, pair plot and heat maps.
- **scikit-learn** for train_test_split, LabelEncoder, StandardScaler, LogisticRegression, KNeighborsClassifier, GaussianNB, SVC and all the evaluation metrics.
- **Microsoft Word** for writing this report.

1.5 How This Report Is Organised

Chapter 2 describes where the data comes from and what each column means. Chapter 3 follows the exploratory data analysis from the first look at the file to the final, scaled feature matrix, and answers Objectives 1 to 3. Chapter 4 contains five analyses: four models, one after the other, and then a comparison of all four that answers Objectives 4 and 5. Each analysis is written in five parts: an introduction, a general description of how the method works, the functions and formulas involved, the results and the visualisation. Chapter 5 gives the conclusion and the limitations, Chapter 6 lists ideas for future work and Chapter 7 gives the references.

2. Source of Dataset

2.1 Where the Data Comes From

The data is the publicly available Telco Customer Churn sample data set that IBM published for its Cognos Analytics tool and that is widely shared on open data platforms such as Kaggle. It describes the customers of a fictitious telecom company that provides home phone and internet services in California. Anyone can download the same file to repeat this work.

Dataset source link: [paste the dataset link here]

The file I used is called telco.csv. It holds one row per customer, 7,043 rows in all, and each row is that customer's record together with the final outcome. The column Churn Label records whether the customer left (Yes) or stayed (No). I did not collect or modify this data myself apart from the cleaning steps described in Chapter 3.

One thing worth saying openly: this is a teaching data set published by IBM, not a raw export from a real operator. It contains no real personal information, which makes it safe to use, but it also means that the accuracy figures in this report show how well these methods work on this kind of data, not a promise of how they would behave on the records of a real telecom company.

2.2 What Each Column Means

There are 50 columns. One is a customer identifier with no predictive meaning, one is the target, and the other 48 describe the customer or the outcome. Table 1 lists all of them. I grouped them loosely by the part of the customer's profile they describe. Some of them turned out to be useless or dangerous for modelling, and Chapter 3 explains which columns I removed and why.

Table 1: *Feature dictionary of the Telco churn dataset (50 columns)*

Column	Category	Type	What it records
Customer ID	Identifier	Text	Unique ID of the customer (dropped before modelling)
Gender	Demographic	Categorical	Male or Female
Age	Demographic	Integer	Age of the customer in years
Under 30	Demographic	Categorical	Yes if the customer is under 30 (derived from Age)
Senior Citizen	Demographic	Categorical	Yes if the customer is 65 or older (derived from Age)
Married	Demographic	Categorical	Marital status, Yes or No

Column	Category	Type	What it records
Dependents	Demographic	Categorical	Whether the customer lives with dependents
Number of Dependents	Demographic	Integer	Count of dependents living with the customer
Country	Location	Text	Country (the same for every row)
State	Location	Text	State (the same for every row)
City	Location	Text	City of residence
Zip Code	Location	Integer	Postal code of the customer
Latitude	Location	Float	Latitude of the customer location
Longitude	Location	Float	Longitude of the customer location
Population	Location	Integer	Population of the zip code area
Quarter	Time	Text	Quarter of the observation (the same for every row)
Referred a Friend	Relationship	Categorical	Whether the customer has referred a friend
Number of Referrals	Relationship	Integer	How many friends the customer has referred
Tenure in Months	Relationship	Integer	Months the customer has stayed (1 to 72)
Offer	Relationship	Categorical	Last promotional offer, A to E (blank means none)
Phone Service	Phone	Categorical	Whether the customer has home phone service
Avg Monthly Long Distance Charges	Phone	Float	Average monthly long distance charge
Multiple Lines	Phone	Categorical	Whether the customer has more than one line
Internet Service	Internet	Categorical	Whether the customer has internet service
Internet Type	Internet	Categorical	DSL, Fiber Optic or Cable (blank means none)
Avg Monthly GB Download	Internet	Integer	Average monthly download volume in GB
Online Security	Add-on	Categorical	Online security add-on, Yes or No
Online Backup	Add-on	Categorical	Online backup add-on, Yes or No
Device Protection Plan	Add-on	Categorical	Device protection add-on, Yes or No
Premium Tech Support	Add-on	Categorical	Premium technical support, Yes or No
Streaming TV	Add-on	Categorical	Streams TV through the service
Streaming Movies	Add-on	Categorical	Streams movies through the service
Streaming Music	Add-on	Categorical	Streams music through the service
Unlimited Data	Add-on	Categorical	Has an unlimited data plan
Contract	Contract	Categorical	Month-to-Month, One Year or Two Year
Paperless Billing	Billing	Categorical	Whether the bill is paperless
Payment Method	Billing	Categorical	Bank Withdrawal, Credit Card or Mailed Check
Monthly Charge	Charges	Float	Current monthly bill of the customer
Total Charges	Charges	Float	Total amount charged so far
Total Refunds	Charges	Float	Total refunds given to the customer
Total Extra Data Charges	Charges	Float	Extra charges for data above the plan
Total Long Distance Charges	Charges	Float	Total long distance charges

Column	Category	Type	What it records
Total Revenue	Charges	Float	Total revenue from the customer (derived from the other charge columns)
Satisfaction Score	Satisfaction	Integer	Survey score from 1 to 5
Customer Status	Outcome	Categorical	Stayed, Churned or Joined
Churn Label	Target	Categorical	Yes if the customer left, No if the customer stayed
Churn Score	Outcome	Integer	A churn score from 0 to 100 already calculated by another model
CLTV	Value	Integer	Customer lifetime value estimate
Churn Category	Outcome	Categorical	Broad category of the reason for leaving
Churn Reason	Outcome	Text	Specific reason given for leaving

2.3 Class Balance and First Impressions

The target column is not evenly split. Out of 7,043 customers, about 1,869 (26.5 percent) left the company and about 5,174 (73.5 percent) stayed. That is a clear imbalance, roughly three stayers for every leaver. It matters because a lazy model that predicts “stayed” for everybody would already be right 73.5 percent of the time without finding a single customer who is about to leave. That 73.5 percent is the number any real model has to beat, and it is the reason this project looks at recall, F1-score and AUC and not at accuracy alone.

A few first impressions from the table of columns. Many columns are Yes/No services, which will need encoding. Four columns (Offer, Internet Type, Contract and Payment Method) have more than two categories. Several columns are blank for good reasons: Offer is blank when the customer took no promotion, and Internet Type is blank when the customer has no internet. Finally, there are columns like Satisfaction Score, Churn Score, Churn Category and Churn Reason which look informative but are recorded after the outcome, and I suspected from the start that they would have to go.

3. EDA Process

This chapter covers everything I did to the data before any model saw it. It follows the first three objectives in order: finding out how many customers leave, finding which groups of customers churn most, and checking whether tenure and monthly charges are related to churn. The last section then turns the cleaned data into the scaled matrix that the models need. I tried to keep the order the same as in the Python script, so that anyone reading the code alongside this report can follow it without jumping around.

3.1 Exploring the Data

> **Objective 1: Find the Percentage of Customers Who Leave**
> Find the percentage of customers who leave the company by counting how many customers churned and how many stayed.

3.1.1 Loading and inspecting the file

The first job was simply to look at the data. I loaded the CSV into a Pandas DataFrame and printed the first and last rows, the shape, the column names, the data types, the summary statistics and the number of missing values in every column. This takes a minute, but it catches a surprising number of problems, such as numbers stored as text or columns that are mostly empty.

Code 1: Loading and inspecting the data

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('telco.csv')

print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
df.info()
print(df.describe())
print(df.isna().sum())
```

The output confirmed 7,043 rows and 50 columns. A few things stood out. The describe() table showed that the numeric columns live on very different scales, with Age in tens, Tenure in Months up to 72 and Total Charges in thousands, so scaling would matter later for KNN and

SVM. The isna().sum() output showed that Offer, Internet Type, Churn Category and Churn Reason contain blanks, and I looked at why in Section 3.1.4.

3.1.2 The target variable

Next I looked at the target column. The counts and percentages come straight from value_counts(), and a pie chart (Figure 1) shows the same thing visually.

Code 2: Counting churn and drawing the pie chart

```
print(df['Churn Label'].value_counts())
print(df['Churn Label'].value_counts(normalize=True) * 100)

plt.figure(figsize=(5, 5))
df['Churn Label'].value_counts().plot.pie(autopct='%1.1f%%')
plt.title("Churned vs Stayed Customers")
plt.ylabel("")
plt.savefig("churn_pie_chart.png")
plt.show()
```

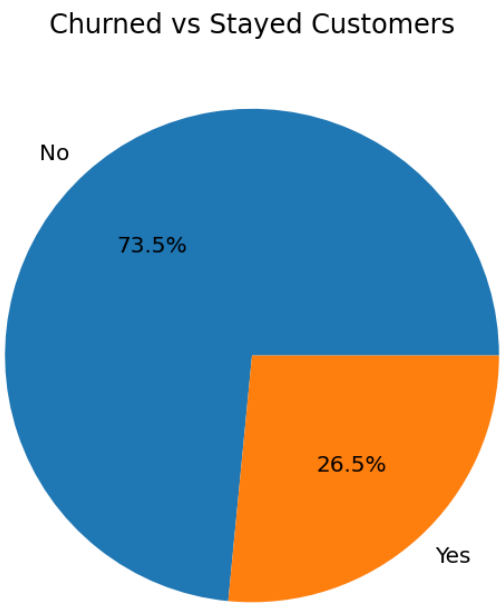


Figure 1: *Churned versus stayed customers*

About 26.5 percent of the customers churned and 73.5 percent stayed. Roughly one customer in four left, which is a high level of attrition for a service business and shows that a retention programme can have a large effect on revenue. It also means that a model which blindly predicts “stayed” for everyone would already be right 73.5 percent of the time. That is the number any real model has to beat.

3.1.3 Distribution of tenure and monthly charge

I drew histograms with 20 bins for the two numeric columns I expected to matter most, Tenure in Months and Monthly Charge. A histogram divides the range of a variable into equal intervals and shows how many customers fall in each, so it reveals whether a variable is symmetric or skewed and whether there are unusual spikes.

Code 3: Histograms of tenure and monthly charge

```
plt.figure(figsize=(8, 4))
plt.hist(df['Tenure in Months'], bins=20, color='purple', edgecolor='black',
alpha=0.9)
plt.title('Histogram of Tenure in Months')
plt.xlabel('Tenure in Months')
plt.ylabel('Frequency')
plt.savefig("hist_tenure.png")
plt.show()

plt.figure(figsize=(8, 4))
plt.hist(df['Monthly Charge'], bins=20, color='orange', edgecolor='black',
alpha=0.9)
plt.title('Histogram of Monthly Charge')
plt.xlabel('Monthly Charge')
plt.ylabel('Frequency')
plt.savefig("hist_monthly_charge.png")
plt.show()
```

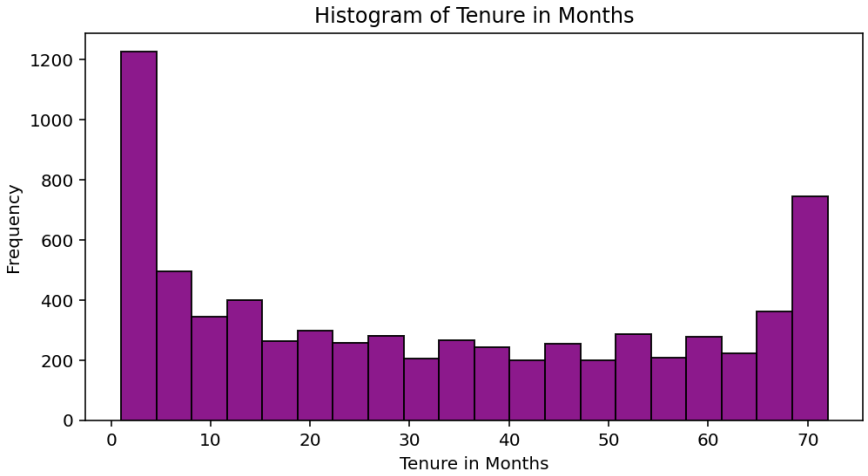


Figure 2: *Distribution of tenure in months*

The tenure histogram (Figure 2) is U-shaped. The first bar is the tallest, with roughly 1,200 customers in their first few months, and there is a second peak of roughly 750 customers at the far right, around 70 to 72 months. In between, every bar holds only about 200 to 400 customers. The company therefore has two very different groups: a large set of new customers and a large set of long-standing loyal ones.

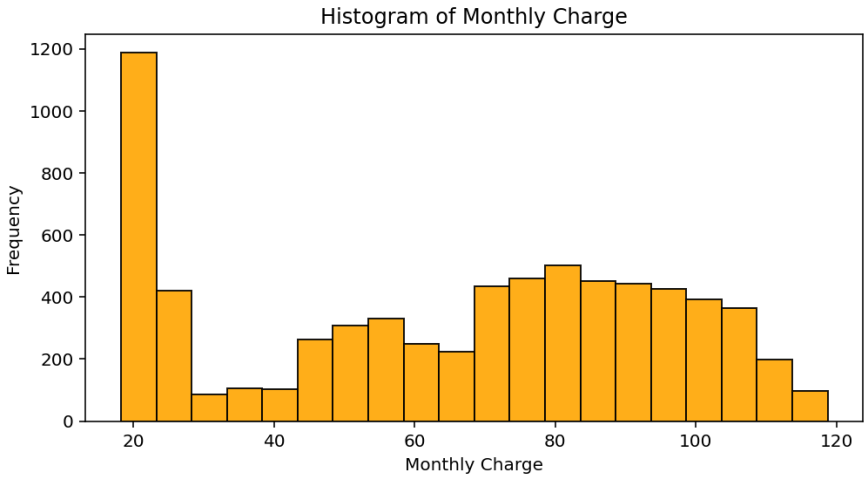


Figure 3: *Distribution of monthly charge*

The monthly charge histogram (Figure 3) is also uneven. A very tall bar of roughly 1,200 customers sits at the lowest charges near 20, there is a dip with only about 100 customers between 30 and 45, and a broad hump between about 70 and 100 peaks at roughly 500 customers near 80. The tall bar on the left is most likely made of customers who take only basic services, while the hump on the right contains customers with internet and add-ons. Neither histogram is bell-shaped, so the median is a safer summary than the mean, which is why I used the median to fill numeric gaps.

3.1.4 Missing values

Missing values can distort statistics and many scikit-learn models refuse to accept them. How to treat a blank depends on why it is blank, and in this data set the reasons are different for different columns, as Table 2 shows.

Table 2: *Missing values per column and how I treated them*

Column(s)	Why the value is blank	Treatment
Offer	The customer has not taken any promotional offer	Filled with the category “None”
Internet Type	The customer has no internet service	Filled with the category “None”
Churn Category, Churn Reason	Customers who did not churn have no reason to record	Dropped later as outcome columns
Age, Tenure in Months, Monthly Charge, Total Charges	No blanks present	Safety step: any blank would be filled with the column median

Code 4: Handling missing values

```
# Blank Offer / Internet Type means "no offer" / "no internet"
df['Offer'] = df['Offer'].fillna('None')
df['Internet Type'] = df['Internet Type'].fillna('None')

# Numeric columns: fill with the median (safe step)
```

```
nums_cols = ['Age', 'Tenure in Months', 'Monthly Charge', 'Total Charges']
for col in nums_cols:
    df[col] = df[col].fillna(df[col].median())
```

The important lesson from this step is that a blank is not always an error. Filling the blanks of Offer and Internet Type with the word "None" keeps those customers in the analysis and creates a meaningful category. "No internet" turned out to be a real and important group, because those customers have the lowest churn rate of all, as Section 3.2 shows.

Objective 1 summary: about 26.5 percent of the customers left the company (roughly 1,869 of 7,043) and 73.5 percent stayed. The target is imbalanced by about three to one, so the baseline accuracy of a model that always says "stayed" is 73.5 percent, and recall, F1-score and AUC must be used alongside accuracy. Tenure and monthly charge are not bell-shaped, and the blanks in Offer and Internet Type mean "none" and were kept as a category.

3.2 Which Customers Churn More

> **Objective 2: Identify the Contract Types with the Highest Churn**
> Identify which contract types have the highest churn rate, and compare churn across internet type, premium tech support, offers and payment method.

3.2.1 Turning churn into a number

To compare groups, I created a helper column named Churn_Num in which No = 0 and Yes = 1. The reason is that the average of a 0/1 column is exactly the share of ones, so the average of Churn_Num inside any group is the churn rate of that group. A Seaborn bar plot with the mean as estimator can then be read directly as a churn rate: a bar of 0.45 means that 45 percent of the customers in that group left. The thin black line on each bar is the confidence interval; a short line means the estimate is precise and a long line means the group is small or the estimate is uncertain.

Code 5: Creating the 0/1 churn helper column

```
le = LabelEncoder()
df['Churn_Num'] = le.fit_transform(df['Churn Label'])   # No = 0, Yes = 1
```

Churn rate of a group = (customers in the group who churned) / (all customers in the group)

3.2.2 Contract type

Contract type shows the strongest difference of all the variables I tried. The same plotting code, shown below, was repeated for the other four variables by changing only the column name.

Code 6: Churn rate by contract (the same pattern is used for the other bar charts)

```
plt.figure(figsize=(8, 4))
sns.barplot(x='Contract', y='Churn_Num', data=df, estimator=np.mean,
color='skyblue')
plt.title("Churn Rate by Contract")
plt.xlabel('Contract')
plt.ylabel('Churn Rate (0.45 = 45%)')
plt.savefig("churn_by_contract.png")
plt.show()

# Same code with x = 'Internet Type', 'Premium Tech Support',
# 'Offer' and 'Payment Method' gives the other four charts.
```

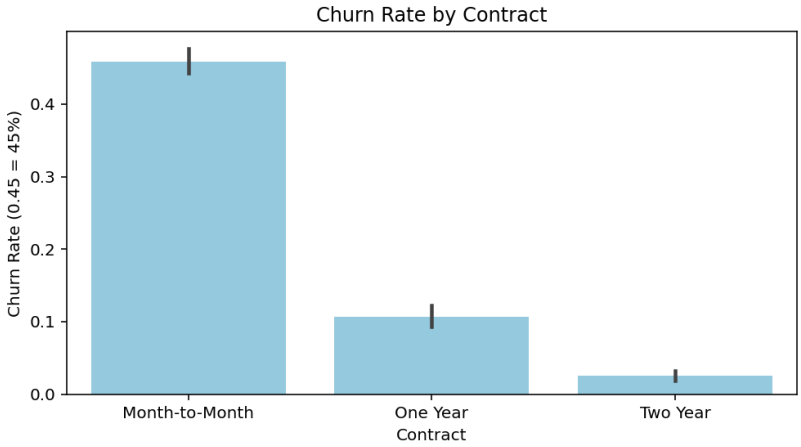


Figure 4: *Churn rate by contract type*

Month-to-month customers churn at about 46 percent, one-year customers at about 11 percent and two-year customers at only about 3 percent (Figure 4). A month-to-month customer is therefore roughly four times as likely to leave as a one-year customer and about eighteen times as likely as a two-year customer. The reason is straightforward: customers without a commitment can leave at any time without penalty, while customers on longer contracts are tied to the company for 12 or 24 months. This answers the main question of Objective 2: **the month-to-month contract has the highest churn rate.**

3.2.3 Internet type

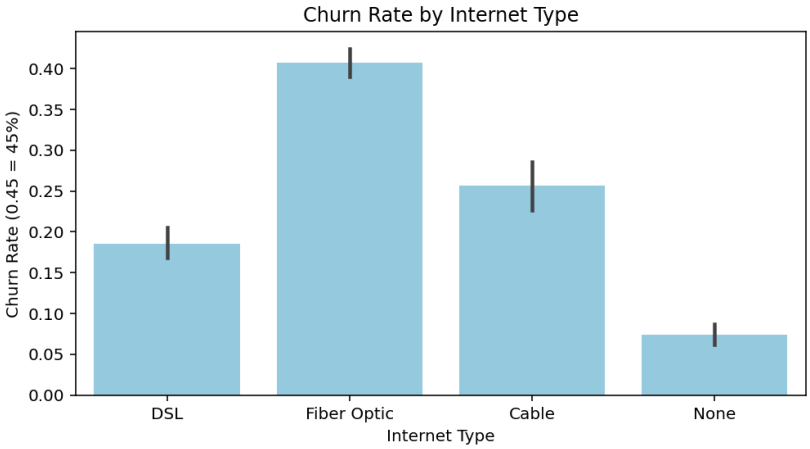


Figure 5: *Churn rate by internet type*

Customers with Fiber Optic internet churn the most, at about 41 percent, followed by Cable at about 26 percent and DSL at about 18.5 percent (Figure 5). Customers with no internet service churn the least, at about 7 percent. Fibre is normally the premium and most expensive product, so its high churn suggests that these customers are price sensitive, may be attracted by competing fibre offers, or may be getting a service that does not match the price they pay.

3.2.4 Premium tech support

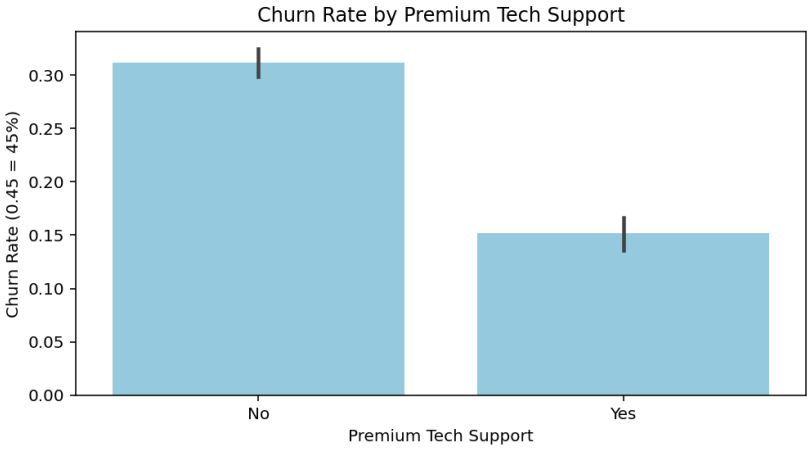


Figure 6: *Churn rate by Premium Tech Support*

Customers without Premium Tech Support churn at about 31 percent, while customers who have it churn at about 15 percent, roughly half the rate (Figure 6). Technical support solves problems quickly and builds a relationship with the customer, which makes it a likely retention factor. A customer who needs help and cannot get it is more likely to leave.

3.2.5 Promotional offer

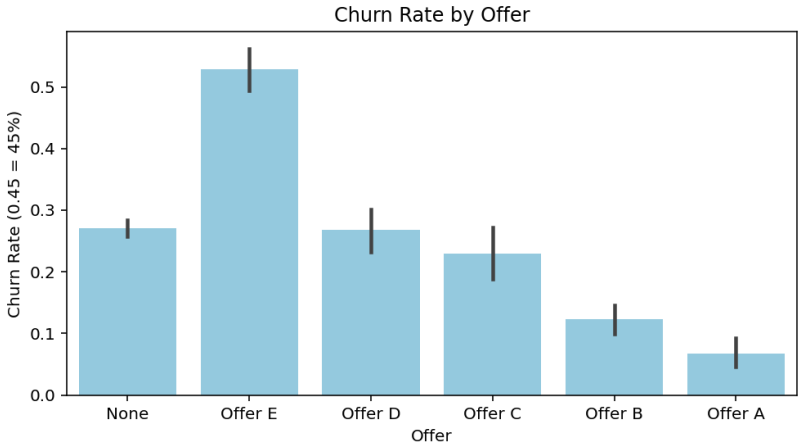


Figure 7: *Churn rate by promotional offer*

The offers differ widely (Figure 7). Customers on Offer E churn at the highest rate of any group in the data, about 53 percent, which is double the rate of customers with no offer (about 27 percent). Offer A (about 7 percent) and Offer B (about 12 percent) have very low churn, while Offers C and D (about 23 and 27 percent) are close to the no-offer level. My reading is that Offer E attracts short-term, bargain-seeking customers who leave when the promotion ends, whereas Offers A and B reach customers who are more loyal.

3.2.6 Payment method

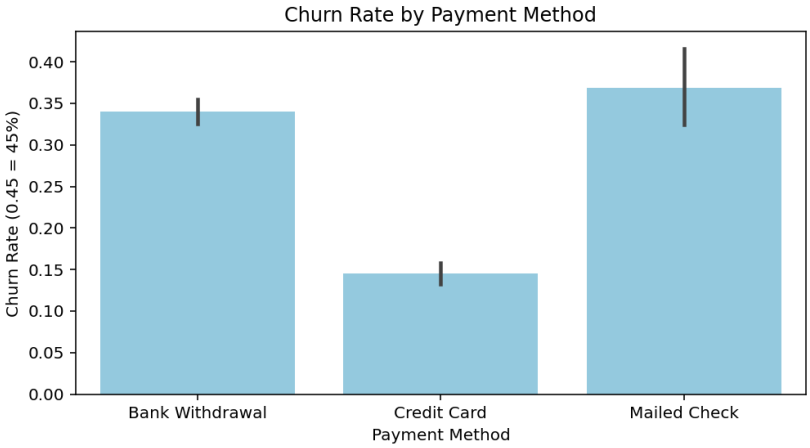


Figure 8: *Churn rate by payment method*

Customers who pay by mailed check (about 37 percent) or bank withdrawal (about 34 percent) churn far more than customers who pay by credit card (about 14.5 percent) (Figure 8). The long confidence interval on mailed check shows that it is a small group, so its exact value is less certain than the others. Automatic card payment probably indicates a smoother and more habitual relationship with the company.

3.2.7 Putting the groups side by side

Table 3: *Approximate churn rate by customer group (read from the bar charts)*

Variable	Highest-churn group	Lowest-churn group
Contract	Month-to-Month (about 46%)	Two Year (about 3%)
Internet Type	Fiber Optic (about 41%)	None (about 7%)
Premium Tech Support	No (about 31%)	Yes (about 15%)
Offer	Offer E (about 53%)	Offer A (about 7%)
Payment Method	Mailed Check (about 37%)	Credit Card (about 14.5%)

Objective 2 summary: the month-to-month contract has the highest churn rate (about 46 percent), far above one-year (about 11 percent) and two-year (about 3 percent) contracts. The typical high-risk customer is on a month-to-month contract with fibre-optic internet, has no premium tech support, pays manually and often holds Offer E.

3.3 Tenure, Monthly Charge and Churn

> **Objective 3: Study Tenure and Monthly Charges**
> Investigate whether customer tenure and monthly charges are related to churn.

3.3.1 Box plots of tenure and monthly charge

Bar charts suit categories, so for the two numeric columns I used box plots. A box plot shows the minimum, first quartile, median, third quartile and maximum of a variable. The box holds the middle half of the customers, the line inside it is the median, the whiskers show the typical range and isolated circles are outliers. Putting the box of churned customers next to the box of stayed customers shows at once whether the two groups differ.

Code 7: Box plots of tenure, monthly charge and monthly charge by contract

```
plt.figure(figsize=(6, 4))
sns.boxplot(data=df, x='Churn Label', y='Tenure in Months')
plt.title("Tenure of Churned and Stayed Customers")
plt.savefig("box_tenure.png")
plt.show()

plt.figure(figsize=(6, 4))
sns.boxplot(data=df, x='Churn Label', y='Monthly Charge')
plt.title("Monthly Charge of Churned and Stayed Customers")
plt.savefig("box_monthly_charge.png")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Contract', y='Monthly Charge', hue='Churn Label')
plt.title("Monthly Charge by Contract (Stayed vs Churned)")
plt.savefig("box_charge_contract.png")
```

```
plt.show()
```

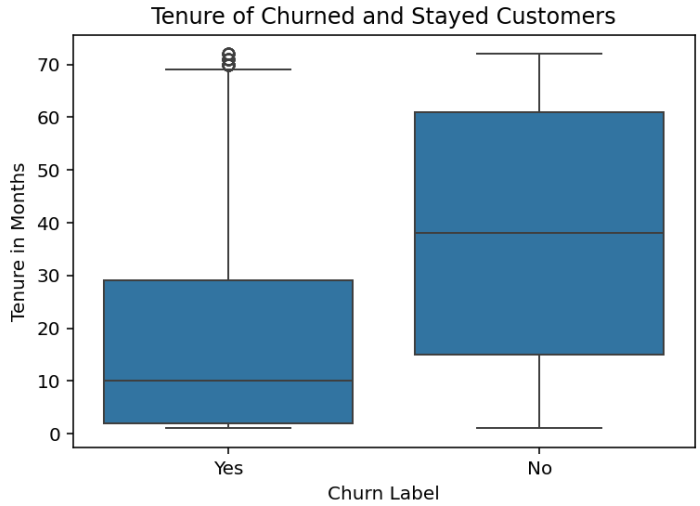


Figure 9: *Tenure of churned and stayed customers*

Churned customers have a much lower tenure (Figure 9). Their median tenure is only about 10 months and half of them left within roughly 2 to 29 months, while customers who stayed have a median of about 38 months and the middle half of them has been with the company for 15 to 61 months. The few circles above the churned box, at about 70 months, are long-standing customers who left, and they are rare. The early months are clearly the dangerous period.

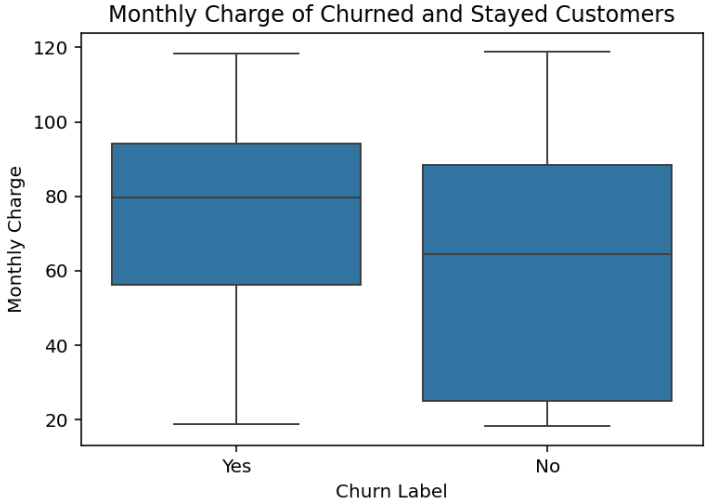


Figure 10: *Monthly charge of churned and stayed customers*

Churned customers also pay more per month (Figure 10). Their median monthly charge is about 80, against about 64.5 for customers who stayed. The box of stayers is much wider and reaches down to about 25, because many loyal customers have cheap plans, whereas churned customers are concentrated between about 56 and 94. Table 4 puts the numbers together.

Table 4: *Approximate box-plot statistics (read from the plots)*

Variable	Group	Q1	Median	Q3
Tenure (months)	Churned (Yes)	about 2	about 10	about 29
Tenure (months)	Stayed (No)	about 15	about 38	about 61
Monthly Charge	Churned (Yes)	about 56	about 80	about 94
Monthly Charge	Stayed (No)	about 25	about 64.5	about 88

3.3.2 Monthly charge within each contract type

A fair question is whether the monthly-charge effect is simply caused by contract type. To check, I plotted the monthly charge for each contract and split every box by churn.

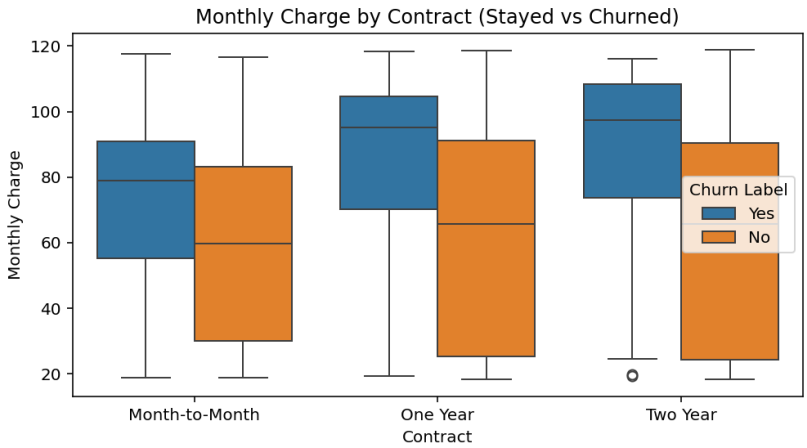


Figure 11: *Monthly charge by contract type (stayed versus churned)*

In all three contract types, the churned customers pay a higher median monthly charge than those who stayed (Figure 11). The median for month-to-month churners is about 79 against about 60 for stayers, and for one-year contracts about 95 against about 66. Two-year churners have a median of about 97, again well above the stayers. The relationship between high charges and churn is therefore not an artefact of contract type, because it holds inside every contract group.

3.3.3 Churn rate across tenure

The box plots compare two groups. To see the pattern month by month, I calculated the churn rate for every tenure value and drew it as a line chart. The same code block also draws the scatter plot and the pair plot used in the next subsection.

Code 8: Line chart, scatter plot and pair plot

```
churn_by_tenure = df.groupby('Tenure in Months')['Churn_Num'].mean() * 100
plt.figure(figsize=(8, 4))
plt.plot(churn_by_tenure.index, churn_by_tenure.values,
         marker='o', linestyle='--', color='b')
```

```
plt.title('Churn Rate (%) by Tenure')
plt.grid()
plt.savefig("line_churn_tenure.png")
plt.show()

sns.scatterplot(x='Tenure in Months', y='Monthly Charge', hue='Churn Label',
                data=df, alpha=0.6, palette='coolwarm')
plt.savefig("scatter_tenure_charge.png")

sns.pairplot(df[['Age', 'Tenure in Months', 'Monthly Charge',
                 'Total Charges', 'Churn Label']], hue='Churn Label')
plt.savefig("pairplot.png")
```

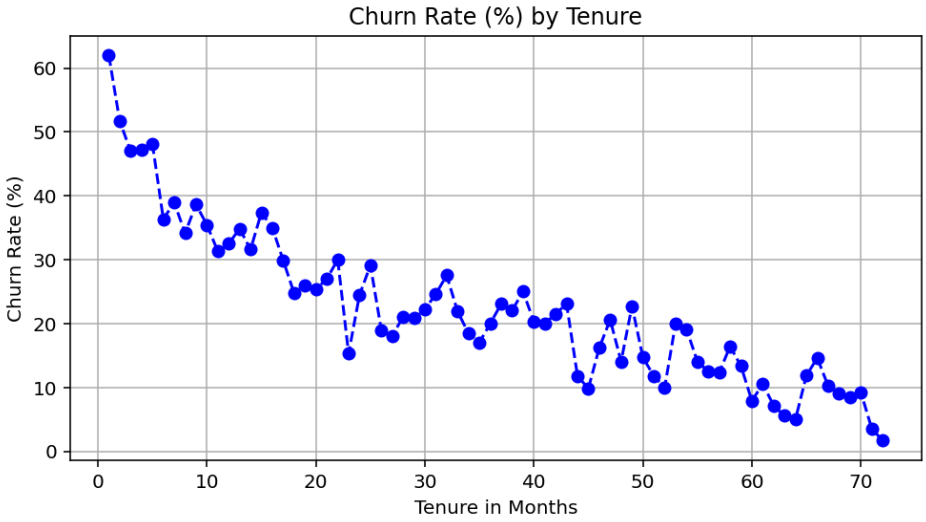


Figure 12: *Churn rate (%) by tenure in months*

This is the clearest picture in the whole analysis (Figure 12). About 62 percent of customers in their first month churn, the rate is still about 47 to 52 percent in months 2 to 5, it falls to about 30 to 35 percent around months 10 to 15, to about 20 percent by months 30 to 40, to about 10 percent by month 60 and to only a few percent in the last months. The line is jagged because some tenure values hold few customers, but the downward trend is unmistakable.

3.3.4 Scatter plot and pair plot

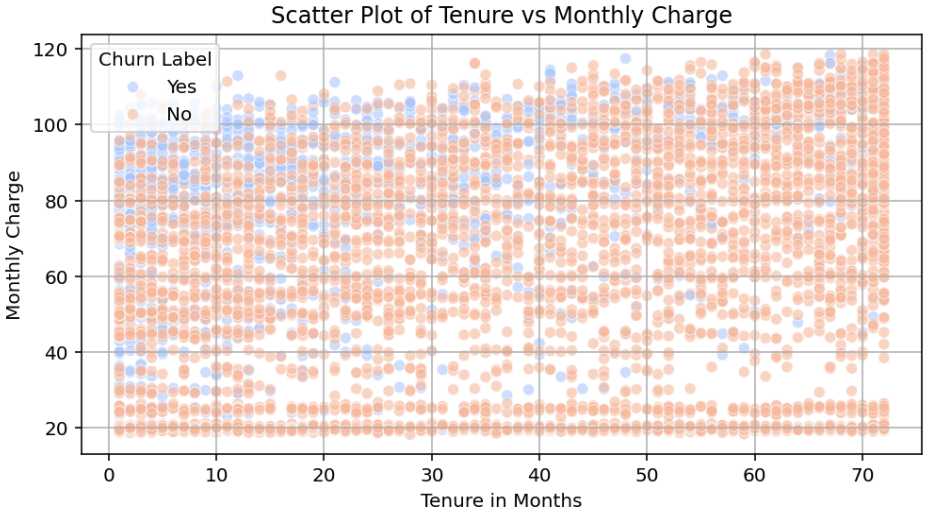


Figure 13: *Scatter plot of tenure versus monthly charge, coloured by churn*

The scatter plot places each customer by tenure and monthly charge (Figure 13). With 7,043 points the plot is crowded and the colours overlap, but two things are still visible. The blue points (churned) are concentrated on the left, at short tenure, and in the upper half of the chart, at monthly charges of roughly 60 to 105. On the right, at long tenure, orange points (stayed) dominate, and the band at the bottom, near a monthly charge of 20, is almost entirely orange. New customers with a high bill are the main risk group, and long-standing customers on cheap plans are the safest.

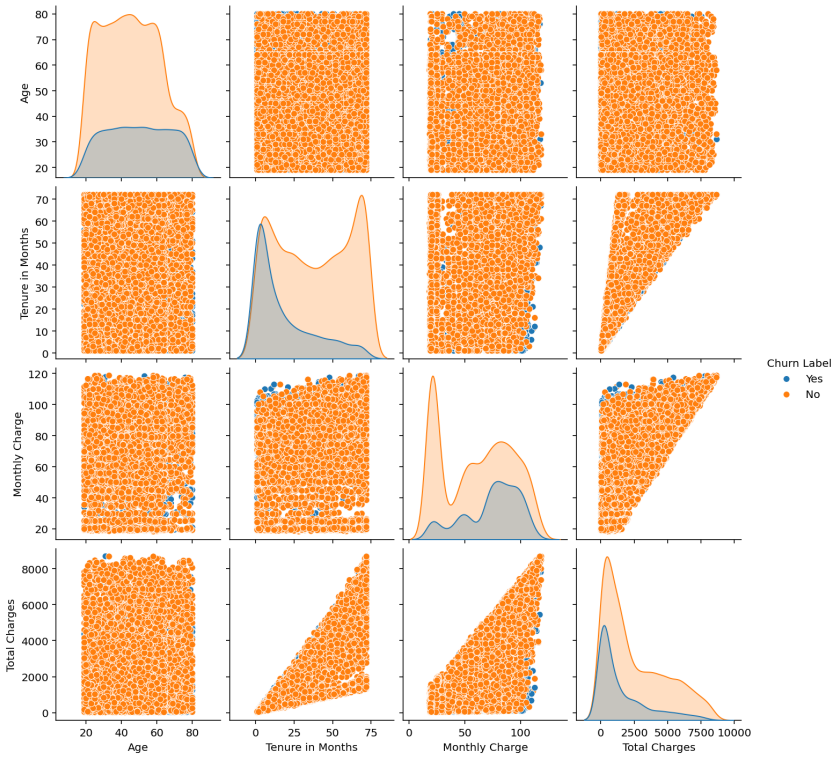


Figure 14: *Pair plot of Age, Tenure, Monthly Charge and Total Charges, coloured by churn*

The pair plot (Figure 14) shows all pairs of the numeric columns together. On the diagonal, the churned tenure curve peaks sharply at the very beginning and then falls away, whereas the stayed curve has a second large peak near 70 months. For monthly charge, the stayed curve has a large peak near 20 and a second hump near 80, while the churned curve has almost nothing near 20 and is concentrated between about 70 and 100. For Age, the churned and stayed curves have almost the same shape, so age on its own tells very little about churn. The off-diagonal panels show the triangular relationship between Total Charges and tenure and between Total Charges and monthly charge.

Objective 3 summary: tenure and monthly charge are both clearly related to churn. Churned customers have a median tenure of about 10 months against about 38 for stayers, and a median monthly charge of about 80 against about 64.5. Low tenure combined with a high monthly bill is the pattern that goes with leaving, and the pattern holds inside every contract type.

3.4 Preparing Features for Machine Learning

With the descriptive questions answered, the remaining job was to turn the table into numbers a model can use, without letting the answer leak into the inputs.

3.4.1 Data leakage and the satisfaction score

Data leakage happens when the inputs given to a model contain information that reveals the answer, or that would not be available when the prediction is needed. Satisfaction Score is the clearest example in this data set. At first it looks like the perfect predictor, so I plotted churn rate against it.

Code 9: Churn rate by satisfaction score and removing the helper column

```
plt.figure(figsize=(8, 4))
sns.barplot(x='Satisfaction Score', y='Churn_Num', data=df,
            estimator=np.mean, color='skyblue')
plt.title("Churn Rate by Satisfaction Score")
plt.savefig("churn_by_satisfaction.png")
plt.show()

df = df.drop(columns=['Churn_Num'])   # remove the helper column
```

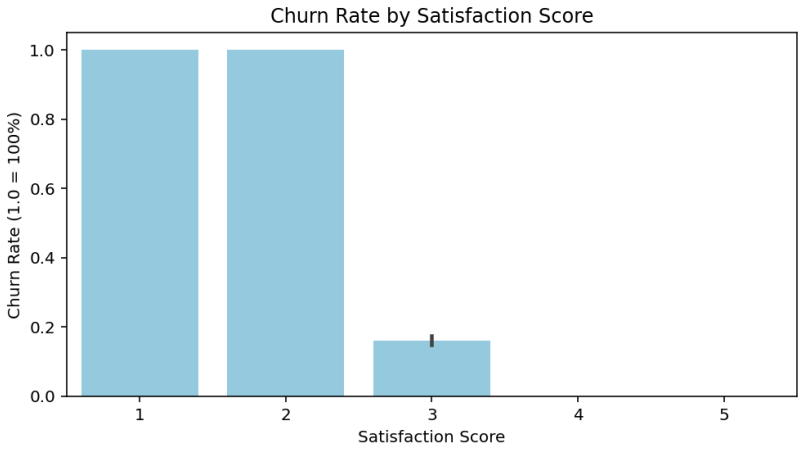


Figure 15: *Churn rate by satisfaction score (1.0 = 100%)*

Table 5: *Churn rate by satisfaction score (read from the bar chart)*

Satisfaction Score	Churn rate
1	100%
2	100%
3	about 16%
4	0%
5	0%

Every customer who gave a score of 1 or 2 churned, and no customer who gave a 4 or 5 did (Figure 15 and Table 5). Real behaviour is never that clean. The score must have been recorded after the customer's decision, so using it would hand the model the answer. A model trained with it would look nearly perfect on the test set and fail in real use, so I decided to remove it,

together with the other columns that describe the outcome. The helper column Churn_Num was also dropped at this point, because keeping it would be a direct leak of the target.

3.4.2 Dropping columns

From the 50 original columns I dropped 19 in three groups, which leaves 30 features and 1 target. Table 6 lists them.

Table 6: *Columns removed before modelling*

Group	Columns	Reason
Identifier and location (8)	Customer ID, Country, State, Quarter, City, Zip Code, Latitude, Longitude	Unique or constant values, or high-cardinality location data with no general predictive value
Outcome columns (5)	Customer Status, Churn Score, Churn Category, Churn Reason, Satisfaction Score	They reveal the outcome (data leakage)
Redundant (6)	Under 30, Senior Citizen, Dependents, Referred a Friend, Internet Service, Total Revenue	Same information already present in another column, for example Under 30 and Senior Citizen come from Age

Code 10: Dropping identifier, leakage and redundant columns

```
drop_id_location = ['Customer ID', 'Country', 'State', 'Quarter',
                    'City', 'Zip Code', 'Latitude', 'Longitude']
drop_leakage = ['Customer Status', 'Churn Score', 'Churn Category',
                'Churn Reason', 'Satisfaction Score']
drop_redundant = ['Under 30', 'Senior Citizen', 'Dependents',
                  'Referred a Friend', 'Internet Service', 'Total Revenue']

df = df.drop(columns=drop_id_location + drop_leakage + drop_redundant)
print("Shape after dropping columns:", df.shape)   # 31 columns
```

Removing redundant columns matters for Logistic Regression, whose coefficients become unstable when features overlap, and for Naive Bayes, which assumes the features are independent.

3.4.3 Encoding categorical columns

Models need numbers, so every text column had to be converted. I used two techniques, depending on the number of categories, as shown in Table 7.

Table 7: *Encoding techniques used*

Technique	Applied to	Reason
LabelEncoder (0/1)	The target Churn Label and 13 two-valued columns: Gender, Married, Phone Service, Multiple Lines, Online Security, Online Backup, Device Protection Plan, Premium Tech Support, Streaming TV, Streaming Movies, Streaming Music, Unlimited Data, Paperless Billing	Only two values, so a 0/1 code creates no false order

Technique	Applied to	Reason
One-hot encoding (get_dummies, drop_first = True)	Offer, Internet Type, Contract, Payment Method	Three or more categories, so label codes would imply a false order (0 < 1 < 2). drop_first removes one dummy column to avoid perfect multicollinearity

Code 11: Encoding the categorical columns

```
df['Churn Label'] = le.fit_transform(df['Churn Label'])

binary_cols = ['Gender', 'Married', 'Phone Service', 'Multiple Lines',
               'Online Security', 'Online Backup', 'Device Protection Plan',
               'Premium Tech Support', 'Streaming TV', 'Streaming Movies',
               'Streaming Music', 'Unlimited Data', 'Paperless Billing']
for col in binary_cols:
    df[col] = le.fit_transform(df[col])

multi_cols = ['Offer', 'Internet Type', 'Contract', 'Payment Method']
df = pd.get_dummies(df, columns=multi_cols, drop_first=True, dtype=int)
df.info()
```

3.4.4 Correlation analysis

After encoding, I drew a correlation heat map of the main numeric columns and the target. Correlation measures how strongly two numeric variables move together, from −1 (opposite directions) to +1 (same direction).

$$r = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{[\Sigma (x_i - \bar{x})^2 \cdot \Sigma (y_i - \bar{y})^2]}$$

Code 12: Correlation heat map

```
corr = df[['Age', 'Tenure in Months', 'Monthly Charge', 'Total Charges',
           'Number of Referrals', 'CLTV', 'Churn Label']].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.savefig("correlation_heatmap.png")
plt.show()
```

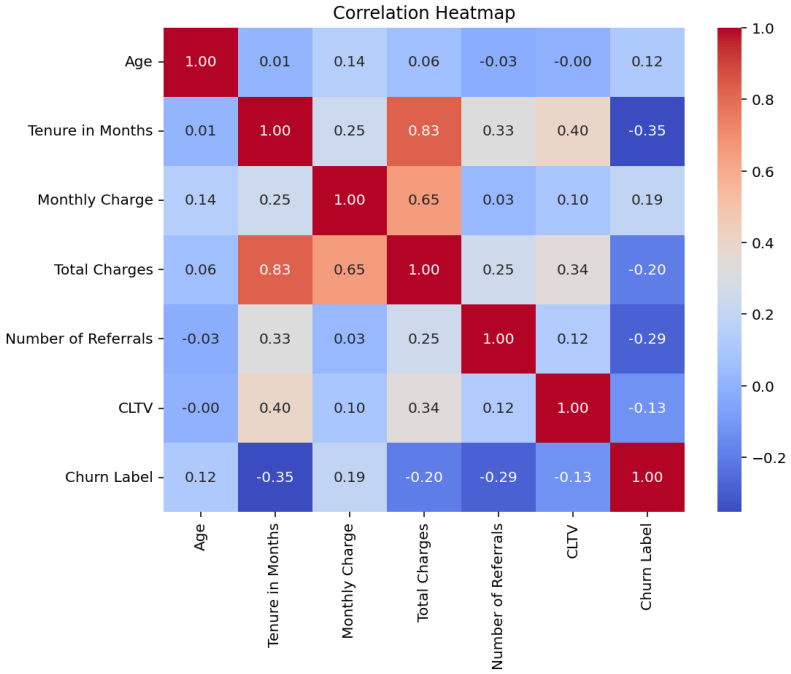


Figure 16: *Correlation heat map of the main numeric columns*

The heat map (Figure 16) confirms what the charts suggested. Total Charges is strongly related to tenure (0.83) and to monthly charge (0.65), which is natural because it is roughly their product. With respect to the target, the strongest relationship is the negative one with Tenure in Months (−0.35): the longer a customer has stayed, the less likely the customer is to churn. Number of Referrals (−0.29), Total Charges (−0.20) and CLTV (−0.13) are also negative, while Monthly Charge (0.19) and Age (0.12) are positive. None of these correlations is very large, which tells me that no single numeric column explains churn on its own and that a model combining many features is needed.

3.4.5 Train and test split

I separated the features (X) from the label (y) and split the data 80:20 with random_state = 42. The argument stratify = y keeps the share of churners almost identical in both parts, which matters for an imbalanced target.

Code 13: Separating features and target, then splitting

```
X = df.drop('Churn Label', axis=1)    # features (input)
y = df['Churn Label']                 # label (output)
print("Number of features:", X.shape[1])

# stratify=y keeps the same churn percentage in train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

The training set has 5,634 customers and the test set has 1,409 customers (1,035 who stayed and 374 who churned). The test set is never used for any decision about the data or the models until the final evaluation.

3.4.6 Scaling with StandardScaler

The features are on very different scales, so I standardised them. StandardScaler subtracts the mean and divides by the standard deviation, so every feature ends up with mean 0 and standard deviation 1:

$$z = (x - \mu) / \sigma$$

Code 14: Scaling the features with StandardScaler

```
Scaler = StandardScaler()
X_train = Scaler.fit_transform(X_train)   # fit on training data only
X_test = Scaler.transform(X_test)         # reuse the same scaling
```

The scaler is fitted on the training data only and then applied unchanged to the test data. Fitting it on the test data would let information from the test set leak into training. Scaling is essential for KNN, because distance would otherwise be dominated by Total Charges, and for SVM, whose margin is distance-based. It also helps Logistic Regression to converge. Naive Bayes is not sensitive to scaling, but I used the same scaled data for all four models so the comparison would be fair.

Preparation summary: 19 columns were dropped (8 identifier and location columns, 5 outcome columns including Satisfaction Score, and 6 redundant columns), 13 two-valued columns and the target were label encoded, four multi-category columns were one-hot encoded, and the data was split 80:20 into 5,634 training and 1,409 test customers and standardised.

4. Analysis on Dataset

With a clean, encoded and scaled data set in hand, the remaining work is to teach models to predict churn and to decide which one does it best. This chapter has five analyses. The first four train one classifier each, and the fifth compares them. Each analysis follows the same five parts: an introduction, a general description of how the method works, the functions and formulas involved, the results and the visualisation.

All four models were trained on the same 5,634 training customers and tested on the same 1,409 test customers, so the comparison is fair. Every metric below is computed on the test customers only. In this project the positive class is "churn", so the terms used in the confusion matrices are:

- **True Positive (TP):** a customer who left and was correctly predicted to leave.
- **True Negative (TN):** a customer who stayed and was correctly predicted to stay.
- **False Positive (FP):** a customer who stayed but was wrongly predicted to leave (a wasted retention offer).
- **False Negative (FN):** a customer who left but was wrongly predicted to stay (a customer lost without any attempt to keep them).

From these four counts, the evaluation metrics used throughout the chapter are defined as follows:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \qquad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN) \qquad F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Accuracy is the share of all predictions that were right. Precision answers "when the model says a customer will leave, how often is it correct?" Recall answers "of the customers who really left, how many did the model find?" The F1-score balances the last two, and I treat it as the most honest single summary here, because accuracy alone can look good on a 73.5 : 26.5 split even for a lazy model. The fifth metric, ROC-AUC, is the area under the curve that plots the true positive rate against the false positive rate as the decision threshold is varied. A value of 0.5 means random guessing and 1.0 means perfect ranking, so AUC measures how well a model puts leavers above stayers regardless of any single threshold.

The baseline for comparison is the model that always predicts "stayed", whose accuracy on the test set is 73.5 percent (1,035 of 1,409 customers).

Objective 4: Predict Customer Churn

Train four classification models (Logistic Regression, KNN, Naive Bayes, SVM) to predict which customers are likely to leave.

4.1 Analysis 1: Logistic Regression

i. Introduction

I started with logistic regression because it is the simplest sensible baseline for a yes or no outcome. If a plain linear model already does well, there is little reason to reach for something complicated, and if it does badly, that tells me the problem has a lot of non-linear structure.

ii. General Description

Logistic regression calculates a weighted sum of all the features and passes it through the logistic (sigmoid) function, which squeezes any number into the range 0 to 1. The result is read as the probability that the customer churns, and a customer with a probability of 0.5 or more is predicted to leave. During training, the weights are chosen to make the observed training labels as likely as possible. scikit-learn also applies L2 regularisation by default, which keeps the weights small and reduces over-fitting.

iii. Specific Requirements, Functions and Formulas

The model is LogisticRegression from scikit-learn with max_iter = 1000, because the default of 100 iterations is not always enough for the optimiser to converge on 30 or more scaled features. The fit() method learns the weights, predict() returns the class and predict_proba() returns the churn probability that I need later for the ROC curve and the AUC.

$$z = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n \qquad P(churn) = 1 / (1 + e^{(-z)})$$

Code 15: Training and evaluating Logistic Regression

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)
prob_log = log_model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))
```

iv. Analysis Results

Of the 1,409 test customers, 1188 were classified correctly, an accuracy of **84.3 percent** and about 11 percentage points above the baseline. The model found **248 of the 374 customers who really left** (recall 0.663) and missed 126. Of the 343 customers it flagged as leavers, 248 really left (precision 0.723) and 95 were false alarms. The F1-score is 0.692 and the AUC is about 0.90, which means the model ranks customers very well.

v. Visualization

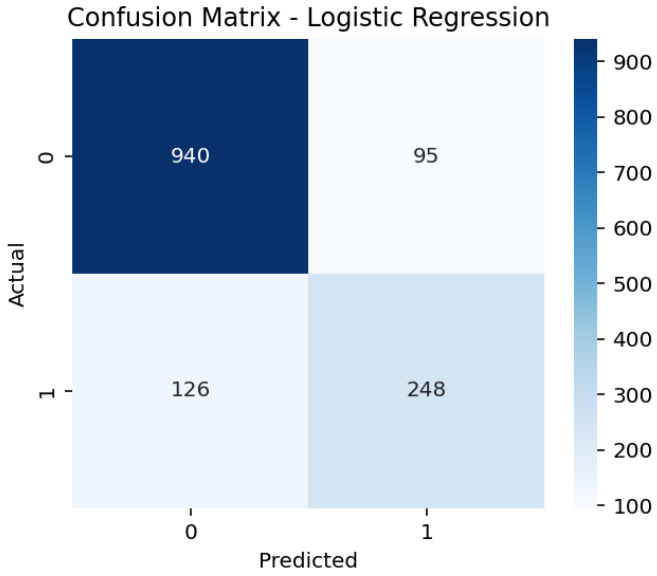


Figure 17: *Confusion matrix of Logistic Regression on the test set*

Figure 17 shows the confusion matrix. Rows are the actual class and columns are the predicted class, with 0 meaning stayed and 1 meaning churned. The dark top-left cell (940) holds the stayers correctly left alone, and the bottom-right cell (248) holds the leavers correctly caught. The two off-diagonal cells, 95 false alarms and 126 missed leavers, are the model's mistakes.

4.2 Analysis 2: K-Nearest Neighbors (KNN)

i. Introduction

KNN is one of the simplest ideas in machine learning and I wanted to see whether "similar customers behave similarly" is enough for churn. It makes no assumption about the shape of the relationship between the features and the target.

ii. General Description

KNN does not learn a formula. It stores the training data, and to predict a new customer it measures the distance to every training customer, takes the K closest ones and lets them vote.

In this project K = 5 and the distance is Euclidean. Because the method depends on distance, all features must be on the same scale, which is why I used StandardScaler. A known weakness is that distance becomes less meaningful when there are many features, as there are here after one-hot encoding, because many 0/1 columns contribute equally to the distance even when they carry little information.

iii. Specific Requirements, Functions and Formulas

The model is KNeighborsClassifier(n_neighbors = 5, metric = “euclidean”). The proportion of the five neighbours that are churners is used as the churn probability for the AUC. I did not tune K in this project.

$$d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$$

Code 16: Training and evaluating KNN

```
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
prob_knn = knn.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

iv. Analysis Results

KNN classified 1120 of 1,409 customers correctly (accuracy **79.5 percent**), which is above the baseline but clearly below Logistic Regression. It found 240 of the 374 leavers (recall 0.642) but also produced 155 false alarms, so its precision is only 0.608. The F1-score of 0.624 and the AUC of about 0.83 are the lowest of the four models.

v. Visualization

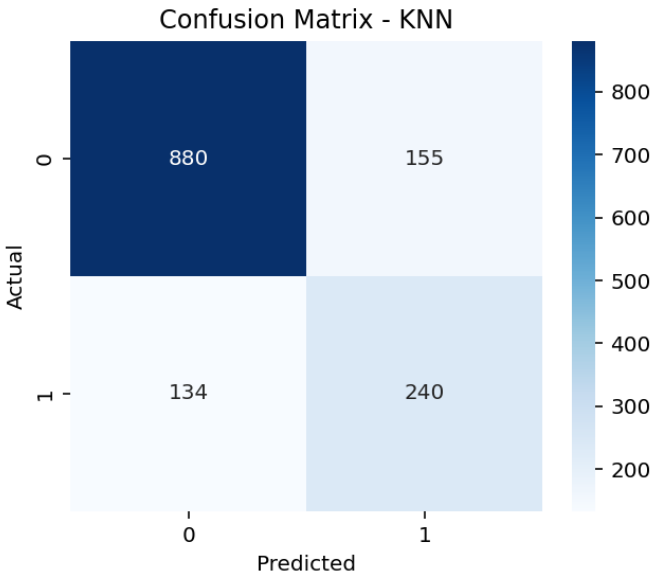


Figure 18: *Confusion matrix of KNN (K = 5) on the test set*

Figure 18 shows that KNN produces 155 false positives and 134 false negatives. Compared with Logistic Regression, it makes 60 more false alarms and misses 8 more leavers, so it is worse on both kinds of mistake.

4.3 Analysis 3: Gaussian Naive Bayes

i. Introduction

Naive Bayes is a probabilistic classifier built on Bayes' theorem. It is extremely fast and is usually a good choice when there is little training data. I included it because it works in a completely different way from the other three models.

ii. General Description

For a given customer, the model calculates, for each class, the prior probability of the class multiplied by the likelihood of each feature value in that class, and chooses the class with the larger result. It is called naive because it assumes that all features are independent once the class is known. The Gaussian version used here also assumes that each feature follows a normal distribution within each class. In this data set many features are 0/1 dummy variables, and several features are strongly related to each other (tenure, monthly charge and total charges), so both assumptions are only roughly true.

iii. Specific Requirements, Functions and Formulas

The model is GaussianNB() with no parameters changed. The predict_proba() method gives the class probabilities from Bayes' rule.

$$P(churn \mid x_1 \ldots x_n) \propto P(churn) \times P(x_1 \mid churn) \times P(x_2 \mid churn) \times \ldots \times P(x_n \mid churn)$$

Code 17: Training and evaluating Naive Bayes

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
prob_nb = nb_model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))
```

iv. Analysis Results

Naive Bayes has a distinctive profile. It has the **highest recall of all four models (0.824)**: it caught 308 of the 374 leavers and missed only 66. It achieved this by flagging 554 customers as leavers, of whom 246 were false alarms, so its precision is only 0.556 and its accuracy of 77.9 percent is the lowest of the four. Its F1-score is 0.664 and its AUC is about 0.87. In practical terms it is an aggressive model: it rarely lets a leaver go unnoticed, but it sends many unnecessary retention offers.

v. Visualization

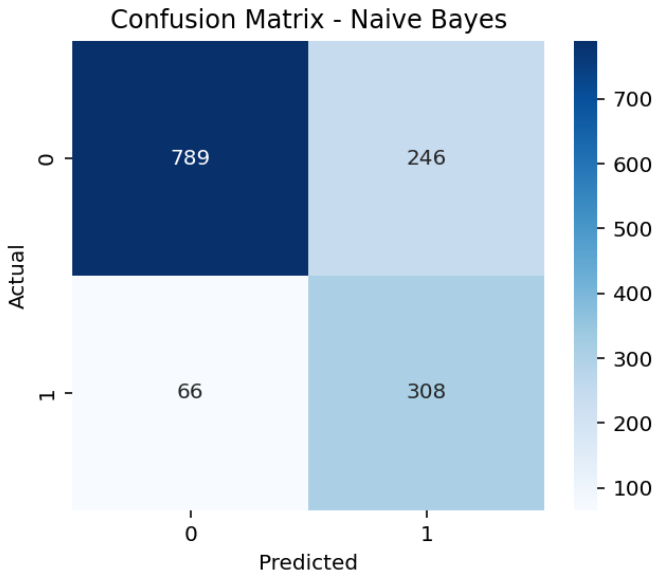


Figure 19: *Confusion matrix of Gaussian Naive Bayes on the test set*

Figure 19 makes the trade-off visible. The bottom-left cell, the missed leavers, is only 66, the smallest of any model, but the top-right cell, the false alarms, is 246, the largest of any model.

4.4 Analysis 4: Support Vector Machine (SVM)

i. Introduction

The support vector machine looks for the best possible boundary between the two classes. I used it because it is strong on data with many features and because it gives a useful comparison with logistic regression, which also learns a linear boundary but by a different principle.

ii. General Description

An SVM finds the separating boundary (a hyperplane) that has the widest possible margin, that is, the largest distance to the nearest training points of each class. These nearest points are called support vectors, and only they decide the boundary. In this project I used the linear kernel, so the boundary is a straight hyperplane in the scaled feature space. A customer is classified by the side of the boundary on which it falls, and the signed distance from the boundary (the decision function) shows how confident the prediction is.

iii. Specific Requirements, Functions and Formulas

The model is SVC(kernel = "linear", random_state = 42). SVC does not give probabilities unless an expensive extra option is switched on, so for the AUC I used decision_function(), which only needs a ranking of the customers.

$$f(x) = w \cdot x + b; \quad \text{predict churn if } f(x) > 0 \qquad \text{Margin} = 2 / ||w||$$

Code 18: Training and evaluating the linear SVM

```
svm_lin = SVC(kernel='linear', random_state=42)
svm_lin.fit(X_train, y_train)
y_pred_svm = svm_lin.predict(X_test)
prob_svm = svm_lin.decision_function(X_test)   # used for AUC

print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

iv. Analysis Results

The linear SVM gave results almost identical to Logistic Regression. It classified 1186 customers correctly (accuracy **84.2 percent**), found 246 of the 374 leavers (recall 0.658) and produced 95 false alarms (precision 0.721). The F1-score is 0.688 and the AUC is about 0.90. The two models differ by only two customers in the false negatives. Two different algorithms

that both learn a straight boundary reaching the same answer suggests that churn can be separated well by a linear combination of the features, and that a more complicated model may bring only a limited gain.

v. Visualization

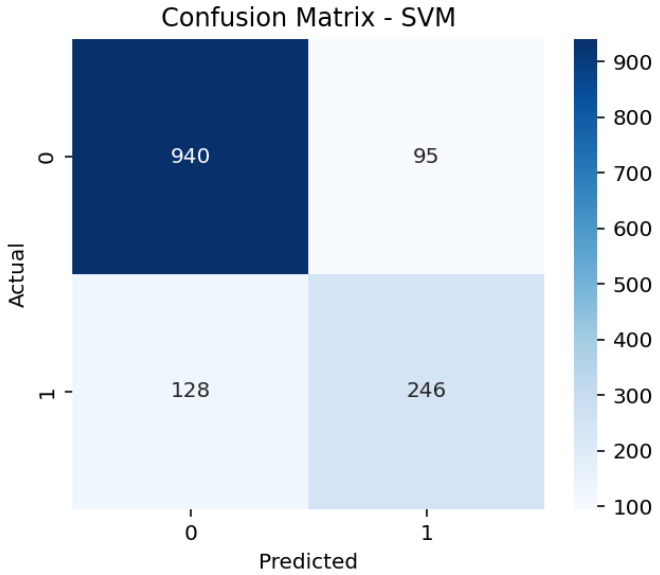


Figure 20: *Confusion matrix of the linear SVM on the test set*

Figure 20 shows 940 true negatives, 95 false positives, 128 false negatives and 246 true positives. The top row is exactly the same as for Logistic Regression.

4.5 Analysis 5: Benchmarking the Four Models

> **Objective 5: Benchmark Model Performance**
> Compare Accuracy, Precision, Recall, F1-score and ROC-AUC of all four models to determine which one predicts churn most effectively.

i. Introduction

With four trained models, the last step is to put their results next to each other and decide which one I would actually recommend to a retention team.

ii. General Description

The comparison collects the test-set metrics of all four models into one table and then plots them so the differences are easy to see. Accuracy is shown, but the decision rests mostly on F1-score, recall and AUC, because a retention team cares about both kinds of error. Missing a

leaver means a lost customer and lost revenue, while flagging a loyal customer wastes an offer and may cost a discount that was not needed.

iii. Specific Requirements, Functions and Formulas

The comparison is a Pandas DataFrame that calls accuracy_score, precision_score, recall_score, f1_score and roc_auc_score for each model. The results are rounded to three decimals, sorted by F1-score and saved to model_comparison.csv. The bar charts are drawn with Seaborn's barplot, and the ROC curves with roc_curve and Matplotlib. The formulas are those given at the start of this chapter.

Code 19: Building the model comparison table

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "KNN", "Naive Bayes", "SVM"],
    "Accuracy":  [accuracy_score(y_test, p) for p in preds],
    "Precision": [precision_score(y_test, p) for p in preds],
    "Recall":    [recall_score(y_test, p) for p in preds],
    "F1 Score":  [f1_score(y_test, p) for p in preds],
    "AUC": [roc_auc_score(y_test, s) for s in scores]})
results = results.round(3)
print(results.sort_values("F1 Score", ascending=False))
results.to_csv("model_comparison.csv", index=False)

print("Baseline accuracy:", round(1 - y_test.mean(), 3))
```

iv. Analysis Results

Table 8 gives the final scores. Accuracy, precision, recall and F1-score were calculated from the confusion matrices; the AUC values were read from the AUC bar chart and are approximate (the exact values are saved in model_comparison.csv).

Table 8: *Final performance of all four models on the 1,409 test customers*

Model	Accuracy	Precision	Recall	F1-score	AUC
Logistic Regression	0.843	0.723	0.663	0.692	about 0.90
KNN	0.795	0.608	0.642	0.624	about 0.83
Naive Bayes	0.779	0.556	0.824	0.664	about 0.87
SVM (linear)	0.842	0.721	0.658	0.688	about 0.90

Table 9: *Confusion matrix counts for each model (test set, 1,409 customers)*

Model	TN	FP	FN	TP
Logistic Regression	940	95	126	248
KNN	880	155	134	240
Naive Bayes	789	246	66	308
SVM (linear)	940	95	128	246

Reading the table metric by metric: Logistic Regression (0.843) and SVM (0.842) are almost equal and clearly best on accuracy, and all four models are above the 0.735 baseline. Logistic Regression and SVM are also the most precise (0.723 and 0.721). Naive Bayes is far ahead on recall (0.824), while the other three lie between 0.642 and 0.663. On F1-score, Logistic Regression is first (0.692), SVM a very close second (0.688), then Naive Bayes (0.664) and KNN (0.624). On AUC, Logistic Regression and SVM tie at about 0.90, Naive Bayes follows at about 0.87 and KNN is last at about 0.83.

It also helps to translate the confusion matrices into what each model would mean for a retention campaign. Table 10 shows how many customers would be contacted, how many real leavers would be reached, and how many offers would be wasted.

Table 10: *What each model would mean for a retention campaign (374 real leavers in the test set)*

Model	Customers contacted	Leavers reached	Wasted offers	Leavers missed
Logistic Regression	343	248	95	126
KNN	395	240	155	134
Naive Bayes	554	308	246	66
SVM (linear)	341	246	95	128

If the company contacts every customer flagged by Logistic Regression, it reaches 248 real leavers with 95 wasted offers. Naive Bayes would reach 60 more leavers than Logistic Regression, but at the price of 151 additional wasted offers. Whether that exchange is worthwhile depends on the economics of the campaign. If an offer is cheap and a saved customer is worth a lot, the higher recall of Naive Bayes is attractive. If offers are costly, the better precision of Logistic Regression and SVM is preferable.

v. Visualization

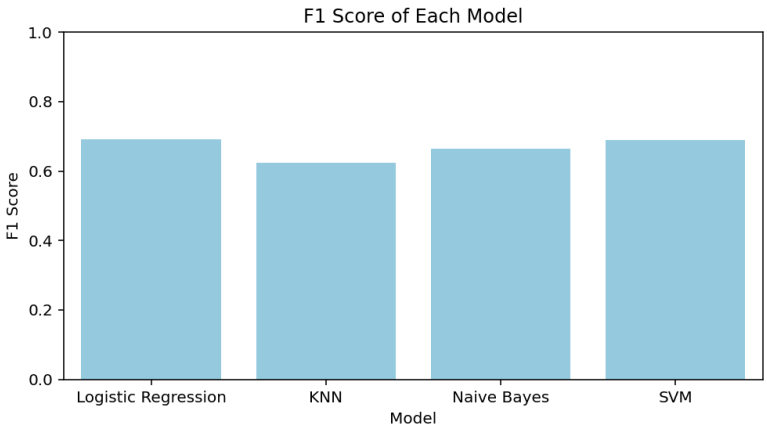


Figure 21: *F1-score of the four models*

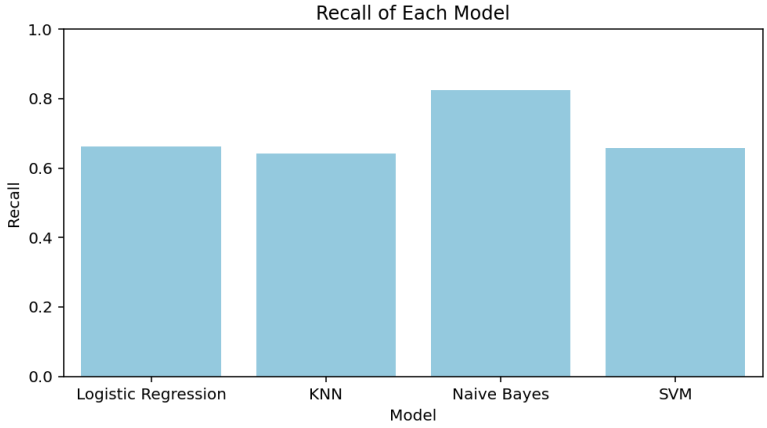


Figure 22: *Recall of the four models*

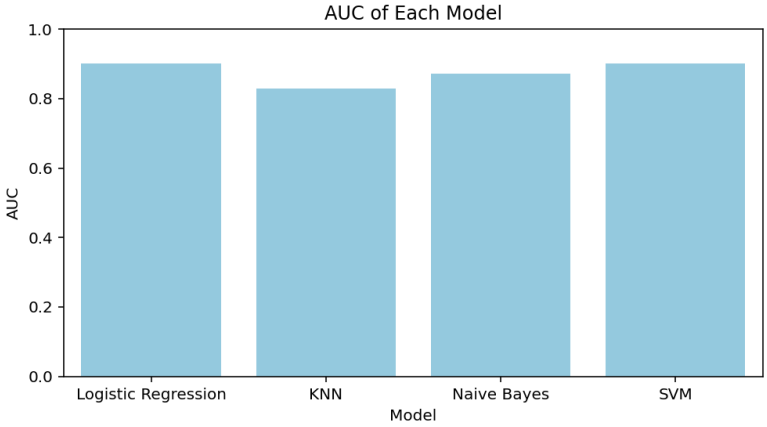


Figure 23: *AUC of the four models*

The three bar charts (Figures 21 to 23) show the same story. On F1 and AUC the four bars are close, with KNN visibly the shortest. On recall, the Naive Bayes bar stands out.

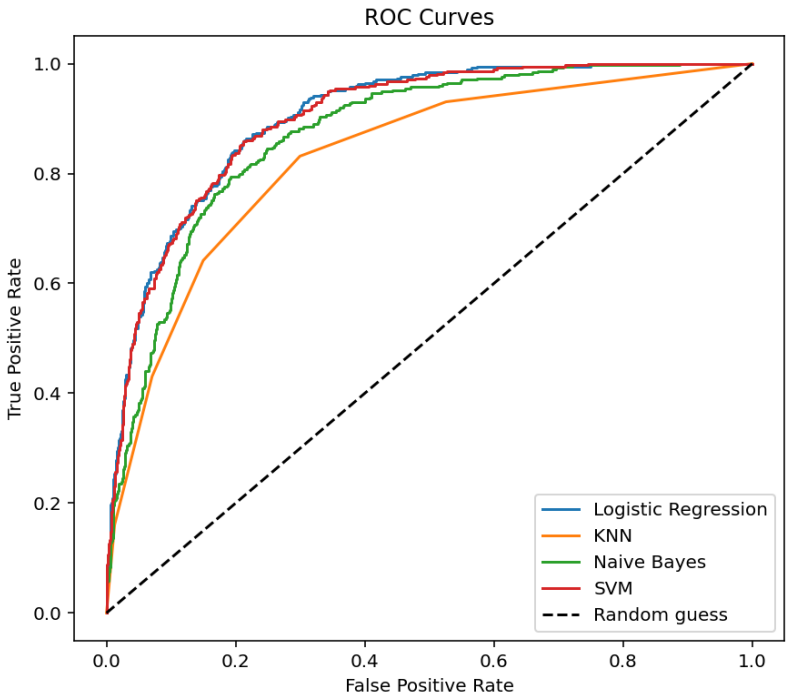


Figure 24: *ROC curves of the four models*

In the ROC plot (Figure 24), the curve that stays closest to the top-left corner belongs to the better model, and the dashed diagonal is random guessing. The curves of Logistic Regression and SVM lie on top of each other and are the highest, which agrees with their equal AUC. The Naive Bayes curve runs clearly below them and the KNN curve is the lowest. The KNN curve is made of a few straight segments instead of a smooth curve, because with K = 5 the model can output only six different probability values (0, 0.2, 0.4, 0.6, 0.8 and 1.0), so it can rank customers only very coarsely. All four curves are far above the random-guess line, so every model has real predictive power.

Why Logistic Regression is the model I would choose

1. **It has the best balance.** Precision (0.723) and recall (0.663) are both solid, so it makes both kinds of mistake rarely.
2. **It has the best accuracy and F1-score,** 84.3 percent and 0.692, on the same test customers every other model saw, and its AUC of about 0.90 is joint highest.
3. **It gives probabilities.** A retention team can see that one customer has a 0.88 chance of leaving and another 0.52, and treat the second as a borderline case.
4. **Its weights can be read.** The coefficients can be inspected and compared with the findings of Chapter 3, so the model's reasoning can be explained to a manager.
5. **It is light.** It trains in a fraction of a second and could sit behind a simple dashboard.

The linear SVM is practically equivalent but slower to train and harder to interpret. Naive Bayes is the right choice only if catching as many leavers as possible matters more than avoiding false alarms. KNN was the weakest on every metric and is not recommended for this data set.

Objective 5 summary: Logistic Regression is the best of the four models on this data, with the highest accuracy, precision and F1-score and a joint-highest AUC. The linear SVM is a very close second, Naive Bayes is third overall but has the highest recall, and KNN is last.

5. Conclusion

5.1 What the Project Found

This project set out to see whether customer churn can be predicted from the data that a telecom company already holds, and which customers are most at risk. Going through the five objectives one at a time, this is what I can say:

1. **Churn percentage (Objective 1).** About 26.5 percent of the customers left and 73.5 percent stayed. The target is imbalanced by roughly three to one, so a model that always says "stayed" already reaches 73.5 percent accuracy, and accuracy alone cannot be trusted.
2. **Contract type and other groups (Objective 2).** Month-to-month customers churn at about 46 percent, against about 11 percent for one-year and about 3 percent for two-year contracts. Churn is also high among fibre-optic customers (about 41 percent), customers without premium tech support (about 31 percent), customers on Offer E (about 53 percent) and customers who pay by mailed check (about 37 percent) or bank withdrawal (about 34 percent).
3. **Tenure and monthly charge (Objective 3).** Both are clearly related to churn. Churned customers have a median tenure of only about 10 months against about 38 for stayers, and they pay a higher median monthly charge (about 80 against about 64.5). The churn rate falls from about 62 percent in the first month to a few percent after five years, and the link with monthly charge holds inside every contract type.
4. **Prediction (Objective 4).** Four classifiers were trained and tested on 1,409 unseen customers. All four beat the 73.5 percent baseline. Logistic Regression and the linear SVM reached about 84 percent accuracy, KNN about 79.5 percent and Naive Bayes about 78 percent.
5. **Benchmark (Objective 5).** Logistic Regression is the best overall model, with accuracy 0.843, precision 0.723, F1-score 0.692 and AUC of about 0.90, followed very closely by the linear SVM (0.842, 0.721, 0.689). Naive Bayes has the highest recall (0.824) but the lowest precision (0.556), and KNN is last on almost every metric.

Two lessons from the process were as important as the numbers. First, Satisfaction Score looked like the best predictor in the whole data set, but it is a leak, because every customer with a score of 1 or 2 had already churned. Removing it, together with Churn Score, Churn

Category, Churn Reason and Customer Status, made the models honest. Second, a blank in a data set is not always an error. The blanks in Offer and Internet Type meant “none”, and keeping them as a category preserved a group of customers who turned out to be the most loyal of all.

5.2 What I Would Tell a Telecom Retention Team

For a retention team, the practical use of this work is a monthly early-warning list. Run the Logistic Regression model on the current customers, sort them by their churn probability, and contact the highest-risk customers first. Because the model gives probabilities, the borderline customers in the middle can be handled next, since they are the ones for whom a small offer is most likely to change the result. If the company is willing to accept more false alarms to catch more leavers, the Naive Bayes list can be used as a second tier.

The data also points to actions that do not need a model at all. Moving customers from month-to-month to a one-year contract is the single most effective lever I found, because the churn rate drops from about 46 percent to about 11 percent. The first year deserves special attention, because the churn rate is highest in the first months and falls steadily afterwards, so welcome calls, early service checks and a first-year loyalty reward are likely to pay off. The fibre-optic experience should be reviewed, since the most expensive product has the highest churn among the internet types. Bundling premium tech support for at-risk customers is a low-cost experiment, because customers with it churn at about half the rate of those without it. Offer E should be re-examined, since customers on it churn at about twice the rate of customers with no offer. Finally, encouraging automatic card payment is worthwhile, since it is associated with the lowest churn of the three payment methods.

5.3 Limitations

I want to be honest about what this work does not show. The data set is a published IBM sample for teaching, so the accuracy numbers say how well these methods do on this kind of data, not how they would do on the records of a real operator. All results come from one train and test split with random_state = 42, so small differences between models, such as the 0.003 gap in F1-score between Logistic Regression and the SVM, should not be over-interpreted. I used mostly default settings: KNN used K = 5 without tuning, the SVM used only the linear kernel and no cross-validation was carried out. The decision threshold was left at the default value, although a company could choose a different threshold to trade precision for recall. I did not

extract the Logistic Regression coefficients into a ranked list of factors, so the statements about which factors matter come from the exploratory charts and not from the model weights. The last point is that churn is also affected by things no table captures, such as competitor promotions, network outages and personal circumstances.

5.4 Project Links

The complete script, figures and this report are kept in a GitHub repository, and a short write up of the project was shared on LinkedIn.

GitHub repository:
https://github.com/RajAnand10/IBM_Telecom_Chrun_Dataset_Predictive_Analysis_Project

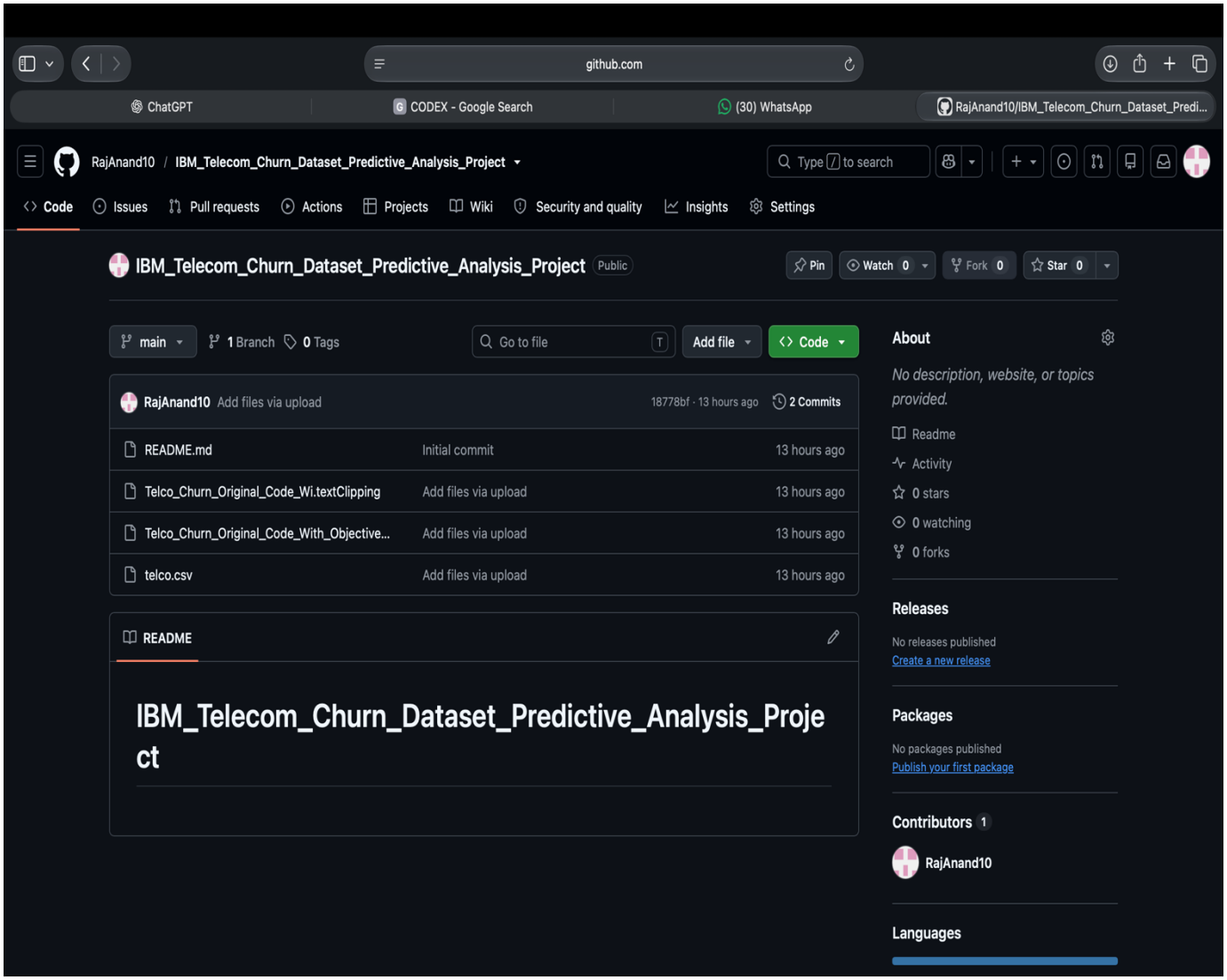


Figure 25: GitHub post sharing the project

LinkedIn post: https://www.linkedin.com/posts/rajanand00_datascience-machinelearning-python-activity-7521629322277879808-BpqV

Figure 25 shows the LinkedIn post in which I shared the project.

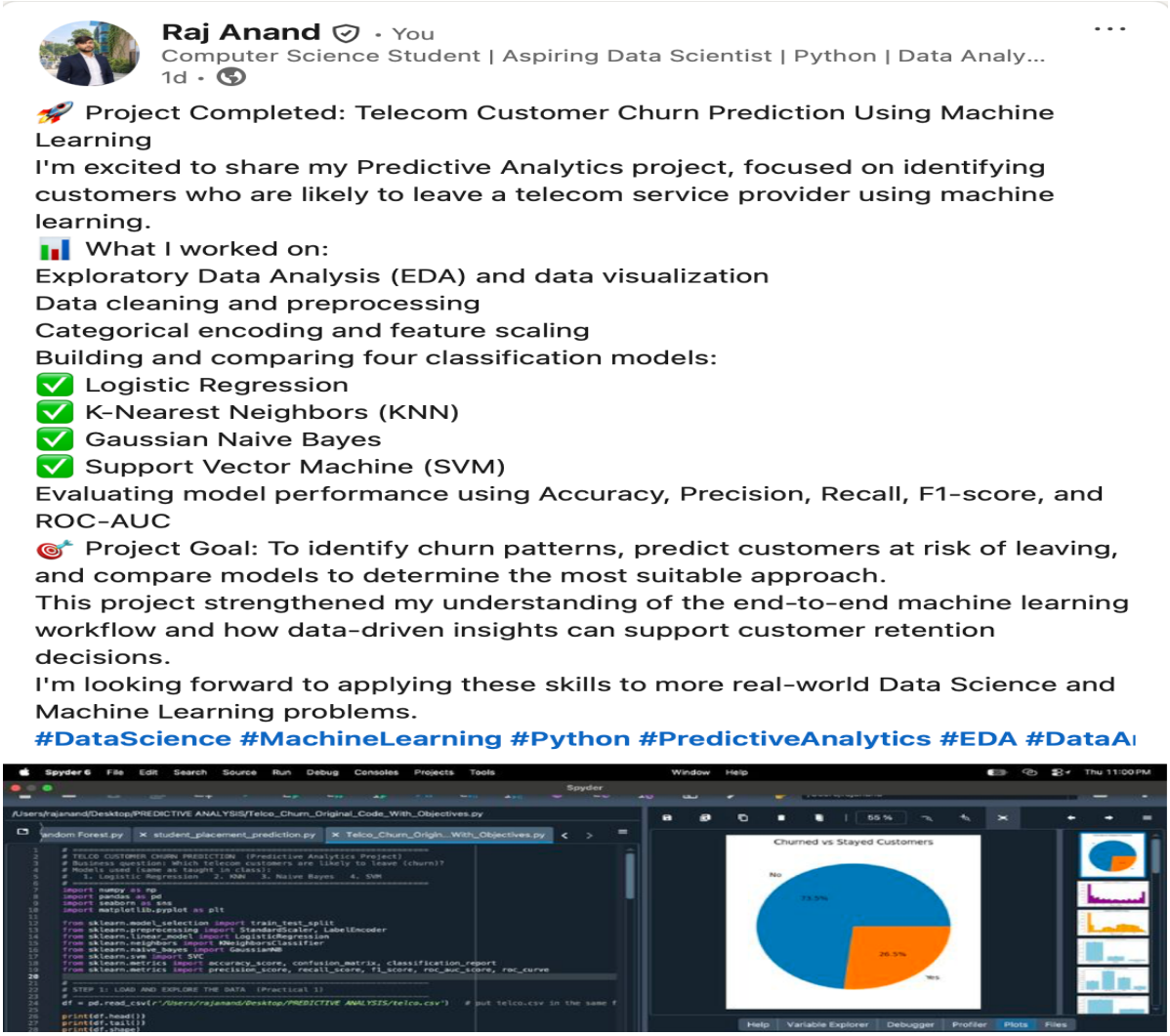


Figure 25: *LinkedIn post sharing the project*

6. Future Scope

This project is a baseline. There are several directions in which it could be taken forward, roughly in the order I would try them.

1. **Fix the weak spots first.** Choose K for KNN properly using cross validation, tune the regularisation of Logistic Regression and the kernel of the SVM, and report results averaged over several random splits instead of one.
2. **Try stronger models.** Random forests and gradient boosted trees (XGBoost, LightGBM) usually do better than single models on tables like this, and they also give feature importances.
3. **Rank the factors.** Extract the Logistic Regression coefficients and plot the five factors that most increase and most reduce churn, so that the exploratory findings can be confirmed by the model.
4. **Handle the class imbalance explicitly.** Class weights, SMOTE oversampling and threshold tuning could raise recall without a large loss of precision.
5. **Use a cost-sensitive view.** Assign a money value to a lost customer and to a retention offer, and choose the threshold that maximises expected profit instead of accuracy.
6. **Explain individual predictions.** Tools such as SHAP could tell the retention team exactly which factors push a particular customer towards leaving.
7. **Build a small web application.** A Streamlit or Flask page where a team uploads a file and receives a ranked list of at-risk customers would turn this project into something people could use.
8. **Test on real data.** The most valuable next step is to retrain on anonymised records from an actual operator over several quarters, and see whether the same factors come out on top.

7. References

[1] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825-2830.

[2] McKinney, W. (2010). Data structures for statistical computing in Python. SciPy Conf., 51-56.

[3] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Comput. Sci. Eng., 9(3), 90-95.

[5] Waskom, M. (2021). seaborn: statistical data visualization. JOSS, 6(60), 3021.

[6] IBM. Telco Customer Churn sample data set (IBM Cognos Analytics sample data), also available on Kaggle.

[7] Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013). Applied Logistic Regression, 3rd ed. Wiley.

[8] Cortes, C. and Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273-297.

[9] Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. IEEE Trans. Inf. Theory, 13(1), 21-27.

[10] Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861-874.